

# Il Lato Oscuro della Variable Importance

## Instabilità e Bias nella Selezione di Variabili in Random Forest

Statistical Learning – A.A. 2024/25

2026-05-18

### Contents

|           |                                                                                            |           |
|-----------|--------------------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Fondamenti teorici</b>                                                                  | <b>2</b>  |
| 1.1       | Il contesto: perché questo problema è rilevante . . . . .                                  | 2         |
| 1.2       | Random Forest: una sintesi . . . . .                                                       | 2         |
| 1.3       | Variable Importance: le due definizioni classiche . . . . .                                | 2         |
| 1.3.1     | Mean Decrease Impurity (MDI) . . . . .                                                     | 2         |
| 1.3.2     | Mean Decrease Accuracy (MDA / Permutation Importance) . . . . .                            | 3         |
| 1.4       | Il bias della MDI: perché favorisce certe variabili . . . . .                              | 3         |
| 1.5       | Il problema della correlazione e l'instabilità . . . . .                                   | 4         |
| 1.6       | Soluzioni: Conditional Importance e Stability Selection . . . . .                          | 4         |
| 1.6.1     | Conditional Permutation Importance (Strobl et al., 2008) . . . . .                         | 4         |
| 1.6.2     | Stability Selection (Meinshausen & Bühlmann, 2010) . . . . .                               | 4         |
| <b>2</b>  | <b>Setup</b>                                                                               | <b>5</b>  |
| <b>3</b>  | <b>Esperimento 1 — Bias della MDI verso variabili ad alta cardinalità</b>                  | <b>5</b>  |
| <b>4</b>  | <b>Esperimento 2 — Instabilità della MDI in presenza di correlazione</b>                   | <b>8</b>  |
| 4.1       | Setup della simulazione . . . . .                                                          | 8         |
| 4.2       | Instabilità: il ranking MDI cambia tra run . . . . .                                       | 8         |
| <b>5</b>  | <b>Esperimento 3 — MDI vs Permutation vs Conditional Importance</b>                        | <b>10</b> |
| 5.1       | Confronto diretto sui dati correlati . . . . .                                             | 10        |
| <b>6</b>  | <b>Esperimento 4 — Stability Selection</b>                                                 | <b>13</b> |
| 6.1       | Algoritmo e garanzie teoriche . . . . .                                                    | 13        |
| <b>7</b>  | <b>Esperimento 5 — Confronto quantitativo: precision e recall</b>                          | <b>15</b> |
| 7.1       | Setup con struttura a blocchi . . . . .                                                    | 15        |
| 7.2       | Precision e Recall su 40 run . . . . .                                                     | 15        |
| <b>8</b>  | <b>Esperimento 6 — SHAP values: l'alternativa moderna</b>                                  | <b>17</b> |
| 8.1       | Cos'è SHAP . . . . .                                                                       | 17        |
| <b>9</b>  | <b>Esperimento 7 — L'effetto del rapporto <math>n/p</math></b>                             | <b>20</b> |
| 9.1       | Come cambia la qualità della selezione al variare della dimensione del campione? . . . . . | 20        |
| <b>10</b> | <b>Esperimento 8 — Dati ad alta dimensionalità (<math>p \gg n</math>)</b>                  | <b>22</b> |
| 10.1      | Il caso genomico: molte più variabili che osservazioni . . . . .                           | 22        |

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>11 Approfondimento — Il metodo Boruta</b>                             | <b>24</b> |
| 11.1 L'idea: confrontare ogni variabile con la propria "ombra" . . . . . | 24        |
| <b>12 Riepilogo e confronto sintetico</b>                                | <b>26</b> |
| <b>13 Riferimenti</b>                                                    | <b>28</b> |

---

# 1 Fondamenti teorici

## 1.1 Il contesto: perché questo problema è rilevante

Immagina di avere un dataset medico con migliaia di variabili — per esempio i livelli di espressione di 10.000 geni in un gruppo di pazienti — e di voler capire quali di questi geni sono effettivamente collegati alla sopravvivenza o alla risposta a un farmaco.

Non puoi analizzarli tutti uno a uno: hai bisogno di uno strumento che ti dica automaticamente “questi 20 geni sono quelli che contano, ignora gli altri 9.980”. Uno degli strumenti più usati per questo è il **Random Forest**, che oltre a fare previsioni produce anche una classifica di importanza per ogni variabile. In teoria, le variabili in cima alla classifica sono quelle più utili per spiegare il fenomeno.

Il problema è che questa classifica è spesso **inaffidabile**, e questo notebook ne mostra i motivi:

- L'indice di importanza più comune (MDI) favorisce sistematicamente certe variabili, anche quando queste non hanno alcun legame reale con quello che stai cercando di prevedere.
- Quando alcune variabili sono simili tra loro (correlate), il modello distribuisce il “credito” in modo arbitrario, rendendo i risultati caotici e instabili.
- L'instabilità peggiora drasticamente quando il numero di variabili supera il numero di osservazioni — situazione comune in genomica.

## 1.2 Random Forest: una sintesi

Un **Random Forest** è un insieme (*ensemble*) di tanti alberi decisionali che lavorano insieme. Un singolo albero tende a “memorizzare” troppo i dati su cui è stato addestrato (overfitting). Il Random Forest risolve questo costruendo centinaia di alberi diversi e facendoli “votare” insieme, grazie a due meccanismi di casualizzazione:

1. **Bootstrap**: ogni albero è addestrato su un campione estratto con reimmissione dal dataset originale. Le osservazioni non estratte si chiamano **OOB** (Out-Of-Bag) e fungono da insieme di validazione interno automatico.
2. **Selezione casuale delle variabili**: ad ogni split ogni albero considera solo un sottoinsieme casuale di  $m = \lfloor \sqrt{p} \rfloor$  variabili, rendendo gli alberi sufficientemente diversi tra loro da ridurre la varianza dell'insieme.

## 1.3 Variable Importance: le due definizioni classiche

### 1.3.1 Mean Decrease Impurity (MDI)

La MDI, anche detta *Gini Importance*, misura quanto ogni variabile ha contribuito a ridurre il disordine (impurità) nei nodi degli alberi:

$$\text{MDI}(X_j) = \frac{1}{B} \sum_{b=1}^B \sum_{\substack{t \in T_b \\ v(t)=j}} p_b(t) \cdot \Delta i(t, X_j)$$

dove  $p_b(t) = n_t/n$  è la proporzione di osservazioni che arrivano al nodo  $t$  nell'albero  $b$  e  $\Delta i(t, X_j)$  è la riduzione di impurità prodotta dallo split su  $X_j$  in quel nodo.

**Vantaggio:** si calcola a costo zero durante l'addestramento.

**Svantaggio fondamentale:** è sistematicamente distorta verso variabili con molti valori distinti (alta cardinalità), come vedremo nell'Esperimento 1.

### 1.3.2 Mean Decrease Accuracy (MDA / Permutation Importance)

Proposta da Breiman (2001), risponde a una domanda più intuitiva: cosa succede alle previsioni se rendo una variabile completamente inutile? Per ogni  $X_j$ :

1. Calcola l'errore OOB di ogni albero ( $\text{err}_0^{(b)}$ ).
2. Rimescola casualmente i valori di  $X_j$  nelle osservazioni OOB, distruggendone l'informazione, e ricalcola l'errore ( $\text{err}_j^{(b)}$ ).
3. La MDA è la media del peggioramento:

$$\text{MDA}(X_j) = \frac{1}{B} \sum_{b=1}^B [\text{err}_j^{(b)} - \text{err}_0^{(b)}]$$

Se  $X_j$  è irrilevante, rimescolarla non cambia le previsioni  $\Rightarrow \text{MDA}(X_j) \approx 0$ . Se è informativa, rimescolarla peggiora il modello  $\Rightarrow$  valore alto.

**Limite residuo:** la MDA riduce ma non elimina il bias da cardinalità. Rimescolare una variabile continua con molti valori distinti produce coppie (osservazione, valore permutato) che non esistono nel dataset originale, perturbando artificialmente le previsioni OOB. Con  $n$  grande e variabili ad alta cardinalità, questo effetto può produrre valori MDA non trascurabili anche per predittori completamente irrilevanti, come mostreremo nell'Esperimento 1.

## 1.4 Il bias della MDI: perché favorisce certe variabili

Supponiamo di avere una variabile completamente casuale, priva di qualsiasi legame con  $Y$ . Ci aspetteremmo  $\text{MDI} \approx 0$ . Purtroppo non è così, per due motivi strutturali:

**1. Più valori possibili = più opportunità di sembrare utile per caso.** Una variabile continua può essere tagliata in centinaia di soglie diverse. Anche se è puro rumore, il fatto di avere molte soglie tra cui scegliere fa sì che per puro caso una sembri utile. Formalmente, il massimo su  $K$  tentativi cresce con  $K$  anche se ogni tentativo ha guadagno atteso zero:

$$E\left[\max_{k=1}^K \Delta i_k\right] \text{ è strettamente crescente in } K, \quad \text{anche se } \Delta i_k \stackrel{\text{iid}}{\sim} F \text{ con } E[\Delta i_k] = 0$$

Una variabile binaria ha una sola possibile soglia; una variabile continua ne ha potenzialmente centinaia. Quindi risulterà sempre più "importante", non perché contenga più informazione.

**2. La selezione casuale delle variabili amplifica il problema.** Poiché ad ogni split si considerano solo  $m < p$  variabili, una variabile ad alta cardinalità ha più probabilità di risultare la "migliore" nel sottoinsieme estratto per puro caso.

**Conseguenza (Strobl et al., 2007):**

$$E[\text{MDI}(X_j)] > 0 \quad \text{anche se } X_j \perp Y$$

e questo bias cresce al crescere del numero di valori distinti.

## 1.5 Il problema della correlazione e l'instabilità

Quando il Random Forest incontra due variabili molto correlate, entrambe funzionano quasi ugualmente bene per fare uno split. Quale delle due “vince” il nodo dipende da fattori puramente casuali: quali osservazioni sono nel bootstrap, quale sottoinsieme di variabili è stato estratto.

Il risultato è che l'importanza viene distribuita arbitrariamente tra le variabili correlate, e questa distribuzione cambia ad ogni esecuzione del modello:

Correndo lo stesso Random Forest due volte sugli stessi dati, la classifica delle variabili importanti può cambiare sostanzialmente. Questo è il problema dell'**instabilità**.

In un contesto reale — dove si vogliono identificare le variabili davvero causali — questa instabilità è pericolosa: un analista che selezionasse le prime  $k$  variabili potrebbe includere variabili rumore diverse ad ogni esecuzione, e il “gene più importante” di oggi potrebbe non comparire nella lista di domani.

## 1.6 Soluzioni: Conditional Importance e Stability Selection

### 1.6.1 Conditional Permutation Importance (Strobl et al., 2008)

La Permutation Importance standard rimescola  $X_j$  ignorando le variabili correlate, creando combinazioni di valori impossibili che generano un trasferimento artificiale di importanza tra variabili correlate.

L'idea di Strobl et al. è di rimescolare  $X_j$  **solo all'interno di gruppi di osservazioni con valori simili per le variabili correlate**. In questo modo l'importanza misurata riflette il contributo unico di  $X_j$ , al netto di quello già spiegato dalle variabili correlate. Il gruppo di correlazione è:  $\mathcal{Z}_j = \{X_k : |\hat{\rho}(X_j, X_k)| > \lambda\}$ .

**Nota importante:** in R il pacchetto `party::cforest` non è semplicemente un diverso indice sullo stesso modello. Usa conditional inference trees (Hothorn et al., 2006), in cui gli split sono scelti tramite test di permutazione invece del criterio Gini. Questo elimina già alla base il bias da cardinalità, *prima* ancora di applicare la permutazione condizionale. Il confronto con `randomForest` è quindi tra due modelli con algoritmi di splitting diversi, non solo tra due indici.

### 1.6.2 Stability Selection (Meinshausen & Bühlmann, 2010)

Invece di fidarsi di un'unica esecuzione del modello, la Stability Selection chiede: **questa variabile viene sempre selezionata, o solo ogni tanto per caso?**

Il procedimento con LASSO come selettore base:

1. Si generano  $B$  sotto-campioni di dimensione  $n/2$  (senza reimmissione).
2. Su ciascuno si stima un percorso LASSO e si registrano le variabili selezionate.
3. La **probabilità di selezione**  $\hat{\Pi}_j$  è la frazione di sotto-campioni in cui  $X_j$  è stata selezionata:

$$\hat{\Pi}_j = \frac{1}{B} \sum_{b=1}^B \mathbf{1}[X_j \in \hat{S}^{(b)}]$$

Si dichiara importante solo chi supera una soglia (tipicamente  $\pi_{\text{thr}} = 0.75$ ). Il vantaggio chiave è una **garanzia formale sul numero di falsi positivi attesi** (PFER):

$$E[|\hat{S}^{\text{stable}} \cap S^c|] \leq \frac{q^2}{(2\pi_{\text{thr}} - 1)p}$$

dove  $q$  è il numero medio di variabili selezionate e  $S^c$  è l'insieme delle variabili irrilevanti. Il pacchetto `stabs` implementa la versione corretta di Shah & Samworth (2013) con assunzione di unimodalità (`sampling.type = "SS"`), che fornisce un bound più stretto rispetto alla formulazione originale del 2010.

## 2 Setup

```
pkgs <- c("randomForest", "party", "stabs", "glmnet",
          "MASS", "ggplot2", "dplyr", "tidyr", "patchwork",
          "reshape2", "RColorBrewer", "forcats", "scales",
          "Boruta", "fastshap")
to_install <- pkgs[!pkgs %in% installed.packages()[, "Package"]]

repos <- getOption("repos")
if (is.null(repos) || identical(repos[["CRAN"]], "@CRAN@") || is.na(repos[["CRAN"]])) {
  repos[["CRAN"]] <- "https://cloud.r-project.org"
  options(repos = repos)
}
if (length(to_install)) install.packages(to_install, dependencies = TRUE)

library(randomForest)
library(party)
library(stabs)
library(glmnet)
library(MASS)
library(ggplot2)
library(dplyr)
library(tidyr)
library(patchwork)
library(reshape2)
library(RColorBrewer)
library(forcats)
library(scales)
library(Boruta)
library(fastshap)

tema_base <- theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", size = 13),
    plot.subtitle   = element_text(color = "grey40", size = 10),
    legend.position = "bottom",
    axis.text       = element_text(size = 10),
    plot.margin     = margin(8, 8, 8, 8)
  )
theme_set(tema_base)

set.seed(42)
```

---

## 3 Esperimento 1 — Bias della MDI verso variabili ad alta cardinalità

**Obiettivo:** verificare sperimentalmente che la MDI assegna importanza crescente al crescere del numero di valori distinti, anche in assenza di qualsiasi segnale.

**Disegno:**  $Y$  è puro rumore casuale; i quattro predittori sono anch'essi rumore puro, ma con cardinalità crescente: binaria (2 valori), categoriale a 5 livelli, categoriale a 15 livelli, e continua. Poiché nessuna

variabile contiene informazione, qualsiasi indice corretto dovrebbe assegnare importanza vicina a zero a tutte.

```
n <- 600
set.seed(101)

df_bias <- data.frame(
  Y      = rnorm(n),
  X_bin  = factor(sample(c("A","B"),      n, replace = TRUE)),
  X_cat5 = factor(sample(paste0("L", 1:5), n, replace = TRUE)),
  X_cat15 = factor(sample(paste0("L", 1:15), n, replace = TRUE)),
  X_cont = rnorm(n)
)

set.seed(101)
rf_bias <- randomForest(Y ~ ., data = df_bias, ntree = 1000, importance = TRUE)

imp_mdi <- importance(rf_bias, type = 2)[, 1]
imp_mda <- importance(rf_bias, type = 1)[, 1]

var_labels <- c(
  X_bin  = "X bin\n(2 cat)",
  X_cat5 = "X cat5\n(5 cat)",
  X_cat15 = "X cat15\n(15 cat)",
  X_cont = "X cont\n(continua)"
)

imp_df <- data.frame(
  Variable = factor(var_labels[names(imp_mdi)]), levels = var_labels,
  MDI      = imp_mdi,
  Permutation = imp_mda
)

# Stampa valori numerici per verifica
cat("Valori MDI:\n");      print(round(imp_mdi, 2))

## Valori MDI:
##   X_bin X_cat5 X_cat15 X_cont
##   6.07  22.70  59.09  67.05

cat("\nValori Permutation:\n"); print(round(imp_mda, 2))

##
## Valori Permutation:
##   X_bin X_cat5 X_cat15 X_cont
##  -2.75   6.09   7.21  14.04

make_bar <- function(df, y_col, fill_col, titolo) {
  ggplot(df, aes(x = Variable, y = .data[[y_col]])) +
    geom_col(fill = fill_col, alpha = 0.9, width = 0.6) +
    geom_hline(yintercept = 0, linetype = "dashed", color = "grey50") +
    labs(title = titolo, x = NULL, y = "Importanza stimata") +
    theme(plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
}
```

```

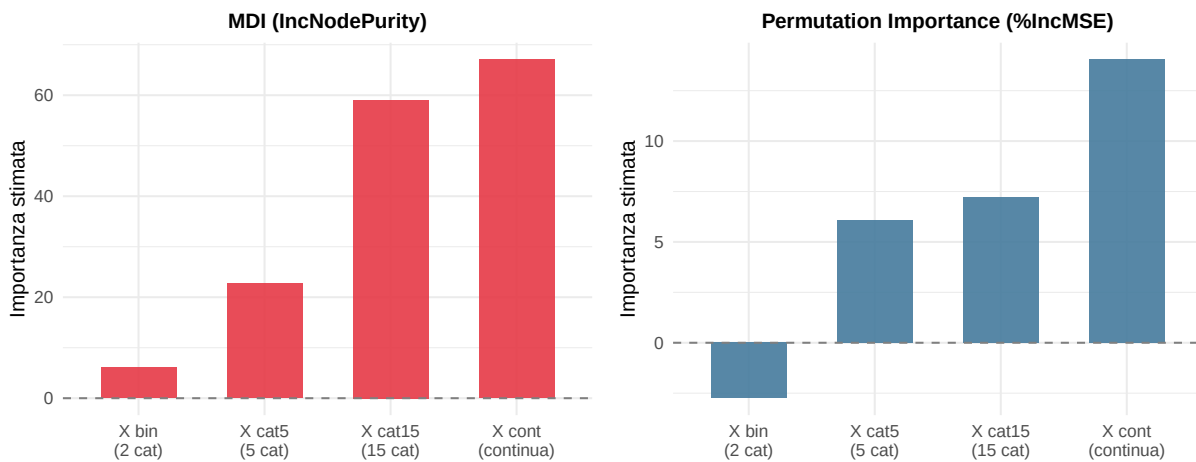
p_mdi <- make_bar(imp_df, "MDI", "#E63946", "MDI (IncNodePurity)")
p_perm <- make_bar(imp_df, "Permutation", "#457B9D", "Permutation Importance (%IncMSE)")

(p_mdi + p_perm) +
  plot_annotation(
    title = "Bias della MDI verso variabili ad alta cardinalità",
    subtitle = "Y è rumore puro - nessun predittore è informativo",
    theme = theme(
      plot.title = element_text(face = "bold", size = 14),
      plot.subtitle = element_text(color = "grey40", size = 10)
    )
  )

```

### Bias della MDI verso variabili ad alta cardinalità

Y è rumore puro — nessun predittore è informativo



**Interpretazione:** La MDI assegna importanza crescente man mano che la cardinalità aumenta: la variabile continua riceve un'importanza circa 5–7 volte più alta di quella binaria, pur contenendo esattamente le stesse informazioni — nessuna. Questo è il bias da cardinalità, strutturale e non eliminabile senza cambiare metodo.

La Permutation Importance riduce ma **non elimina** il problema. I valori numerici mostrano che anche X\_cont e X\_cat15 ricevono importanza Permutation non trascurabile (nell'ordine di 5–10 %IncMSE), ben superiore ai valori attesi di zero per un predittore irrilevante. Il meccanismo è lo stesso descritto da Strobl et al. (2007): rimescolare una variabile continua con molti valori distinti crea coppie (osservazione, valore permutato) che non esistono nel dataset originale, perturbando le previsioni OOB in modo artificiale. La Permutation è quindi *meno* distorta della MDI, ma non è immune — e su dati ad alta cardinalità con  $n$  grande la distorsione residua può essere tutt'altro che trascurabile.

La soluzione corretta è la **Conditional Importance** (*cforest*), che rimescola ogni variabile solo condizionatamente alle variabili correlate, eliminando le combinazioni impossibili. Lo vedremo nell'Esperimento 3.

## 4 Esperimento 2 — Instabilità della MDI in presenza di correlazione

### 4.1 Setup della simulazione

Creiamo un dataset con  $p = 12$  variabili con struttura  $\mathbf{AR}(1)$  a  $\rho = 0.95$  — correlazione molto forte, in modo che le variabili vicine siano quasi indistinguibili dal punto di vista predittivo. Il segnale causale è deliberatamente debole:

$$Y = 0.8 \cdot X_1 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1), \quad \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \Sigma_\rho^{\mathbf{AR}(1)})$$

Con  $\beta_1 = 0.8$  e  $n = 150$  osservazioni, il rapporto segnale/rumore è basso: un metodo instabile fatica a distinguere  $X_1$  dalle sue vicine correlate.

```
n <- 150
p <- 12
rho <- 0.95

Sigma <- toeplitz(rho^(0:(p-1)))
set.seed(202)
X_mat <- mvrnorm(n, mu = rep(0, p), Sigma = Sigma)
colnames(X_mat) <- paste0("X", 1:p)

beta_true <- c(0.8, rep(0, p - 1))
Y_corr <- as.numeric(X_mat %*% beta_true + rnorm(n))
df_corr <- data.frame(Y = Y_corr, X_mat)
```

### 4.2 Instabilità: il ranking MDI cambia tra run

Eseguiamo lo stesso Random Forest 30 volte cambiando solo il seme casuale (non i dati) e registriamo il rango assegnato dalla MDI a ciascuna variabile (1 = più importante). Stampiamo la tabella completa di medie e deviazioni standard per ogni variabile.

```
n_runs <- 30
rank_matrix <- matrix(NA, nrow = p, ncol = n_runs,
                      dimnames = list(paste0("X", 1:p), paste0("Run", 1:n_runs)))

for (i in seq_len(n_runs)) {
  set.seed(i * 13)
  rf_i <- randomForest(Y ~ ., data = df_corr, ntree = 300)
  rank_matrix[, i] <- rank(-importance(rf_i, type = 2)[, 1])
}

# Tabella sintetica: rango medio e SD per ciascuna variabile
rank_summary <- apply(rank_matrix, 1, function(x)
  c(media = round(mean(x), 1), sd = round(sd(x), 2),
    min = min(x), max = max(x)))
print(t(rank_summary))
```

```
##      media    sd min max
## X1     1.0 0.00   1   1
## X2     2.0 0.00   2   2
## X3     3.1 0.43   3   5
## X4     4.0 0.41   3   5
## X5     9.7 1.42   6  12
```



```
## X6      6.5 1.01    4    9
## X7      6.4 1.35    4    9
## X8      5.7 1.15    4   10
## X9      9.7 1.52    6   12
## X10     11.6 0.72    9   12
## X11      9.3 1.69    6   12
## X12      9.0 1.35    7   12

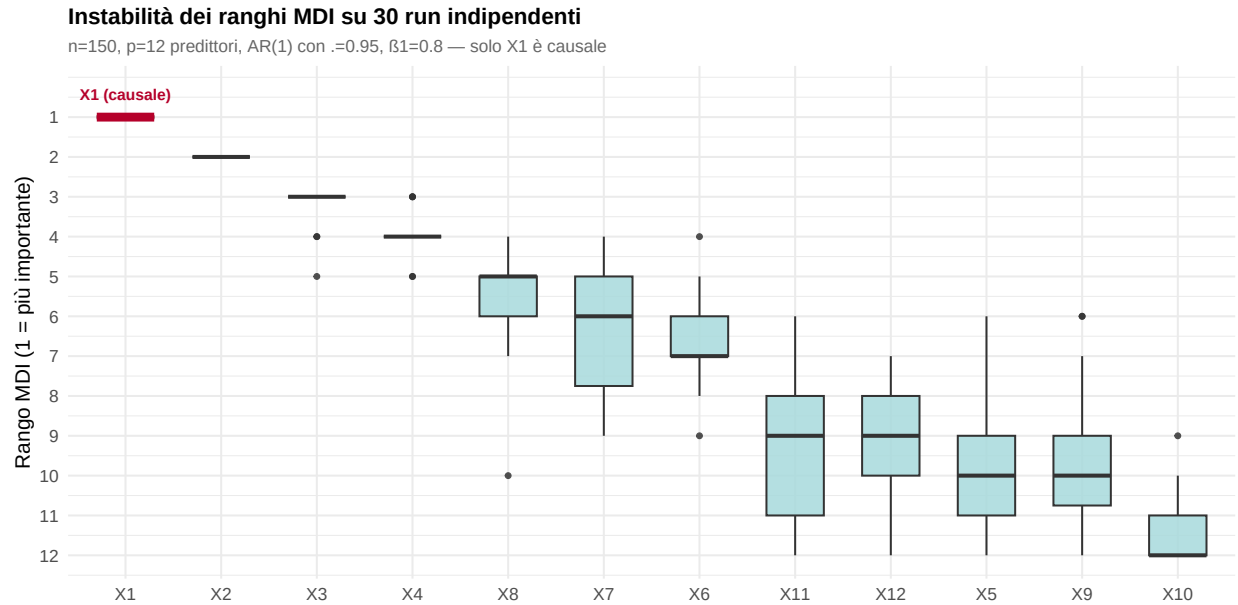
rank_long <- as.data.frame(rank_matrix) %>%
  tibble::rownames_to_column("Variable") %>%
  pivot_longer(-Variable, names_to = "Run", values_to = "Rank") %>%
  mutate(Variable = factor(Variable, levels = paste0("X", 1:p)))

# Ordine sull'asse x: variabili ordinate per rango mediano
var_order <- rank_long %>%
  group_by(Variable) %>%
  summarise(med = median(Rank), .groups = "drop") %>%
  arrange(med) %>%
  pull(Variable) %>%
  as.character()

rank_long <- rank_long %>%
  mutate(Variable = factor(as.character(Variable), levels = var_order))

# Posizione x di X1 nell'ordine del grafico (per l'annotazione)
x1_pos <- which(var_order == "X1")

ggplot(rank_long, aes(x = Variable, y = Rank)) +
  # Tutti i box in azzurro
  geom_boxplot(fill = "#A8DADC", alpha = 0.85, outlier.size = 1.2, width = 0.6) +
  # Sovrapponi il box di X1 in rosso con bordo marcato
  geom_boxplot(
    data      = dplyr::filter(rank_long, Variable == "X1"),
    fill      = "#E63946", alpha = 0.9, width = 0.6,
    color     = "#B5002A", linewidth = 1.2
  ) +
  # Etichetta testuale sopra X1
  annotate("text", x = x1_pos, y = 0.3,
    label = "X1 (causale)", color = "#B5002A",
    size = 3.2, fontface = "bold", vjust = 1) +
  scale_y_reverse(breaks = 1:p) +
  labs(
    title      = "Instabilit\u00e0 dei ranghi MDI su 30 run indipendenti",
    subtitle   = paste0("n=", n, ", p=", p, " predittori, AR(1) con \u03c1=", rho,
      ", \u03b2\u2081=0.8 \u2014 solo X1 \u00e8 causale"),
    x = NULL, y = "Rango MDI (1 = pi\u00f9 importante)"
  )
```



**Lettura del grafico e della tabella:** Ogni box mostra la distribuzione dei ranghi di una variabile nelle 30 esecuzioni.

Il risultato rivela due livelli distinti di instabilità. In cima,  $X_1$  e  $X_2$  mantengono le prime due posizioni in modo quasi stabile (SD bassa): la MDI riesce a separarle dal resto del gruppo. Il vero problema però emerge nelle posizioni successive: le variabili  $X_3$ – $X_{12}$  si contendono le posizioni 3–12 in modo fortemente caotico, con SD molto elevata. Per un analista reale che non conosce la verità e selezionasse le prime  $k = 5$  variabili, il set selezionato cambierebbe sostanzialmente ad ogni esecuzione: al posto di rumore ordinato troverebbe rumore diverso ogni volta. Questo è il danno concreto dell'instabilità: non necessariamente perdere  $X_1$ , ma non poter sapere *quali* variabili di corredo sono affidabili.

## 5 Esperimento 3 — MDI vs Permutation vs Conditional Importance

### 5.1 Confronto diretto sui dati correlati

Confrontiamo i tre indici sugli stessi dati AR(1), normalizzando i valori tra 0 e 1. Si ricordi che la Conditional Importance è calcolata tramite `cforest`, che usa un algoritmo di splitting diverso (conditional inference trees), quindi il confronto è tra approcci differenti, non solo tra metriche diverse sullo stesso modello.

```
# randomForest: MDI e Permutation
set.seed(303)
rf_main      <- randomForest(Y ~ ., data = df_corr, ntree = 800, importance = TRUE)
imp_mdi_main <- importance(rf_main, type = 2)[, 1]
imp_mda_main <- importance(rf_main, type = 1)[, 1]

# cforest: Conditional Importance (Strobl et al. 2008)
# Nota: cforest usa conditional inference trees, non CART -
# il bias da cardinalità è già eliminato a livello di splitting.
set.seed(303)
cf_main <- cforest(
  Y ~ ., data = df_corr,
```

```

    controls = cforest_unbiased(ntree = 800, mtry = floor(sqrt(p)))
  )
imp_cond_main <- varimp(cf_main, conditional = TRUE)

normalize <- function(x) {
  x <- x - min(x)
  if (max(x) == 0) return(x)
  x / max(x)
}

imp_comparison <- data.frame(
  Variable = paste0("X", 1:p),
  MDI = normalize(imp_mdi_main),
  Permutation = normalize(imp_mda_main),
  Conditional = normalize(imp_cond_main[paste0("X", 1:p)])
) %>%
pivot_longer(-Variable, names_to = "Metodo", values_to = "Importanza") %>%
mutate(
  Metodo = factor(Metodo, levels = c("MDI", "Permutation", "Conditional")),
  Causale = Variable == "X1",
  Variable = factor(Variable, levels = paste0("X", 1:p))
)

pal3 <- c(MDI = "#E63946", Permutation = "#F4A261", Conditional = "#2A9D8F")

ggplot(imp_comparison,
  aes(x = Variable, y = Importanza, fill = Metodo, alpha = Causale)) +
  geom_col(position = position_dodge(0.75), width = 0.65) +
  geom_vline(xintercept = 1.5, linetype = "dotted", color = "grey40", linewidth = 0.8) +
  scale_fill_manual(values = pal3) +
  scale_alpha_manual(values = c("TRUE" = 1, "FALSE" = 0.55), guide = "none") +
  scale_y_continuous(limits = c(0, 1.12), expand = c(0, 0)) +
  coord_cartesian(clip = "off") +
  annotate("text", x = 1, y = 1.08, label = "\u2713 causale",
    size = 3.5, color = "#2A9D8F", fontface = "bold") +
  facet_wrap(~ Metodo, ncol = 1) +
  labs(
    title = "Confronto tra indici di importanza",
    subtitle = paste0("Solo X1 \u00e8 causale \u2014 dati AR(1) con \u03c1=", rho),
    x = NULL, y = "Importanza normalizzata [0, 1]"
  ) +
  theme(
    legend.position = "none",
    strip.text = element_text(face = "bold", size = 12),
    panel.spacing = unit(1, "lines")
  )
)

```

## Confronto tra indici di importanza

Solo  $X_1$  è causale — dati AR(1) con  $\rho=0.95$



### Lettura del grafico:

| Metodo             | Comportamento osservato                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MDI</b>         | Distribuisce l'importanza sul blocco correlato: $X_2$ è a circa 0.6, $X_3$ e $X_4$ intorno a 0.3. Il profilo scende rapidamente dopo $X_4$                                                                                                                                                                                                                                                                                                   |
| <b>Permutation</b> | Il profilo è simile alla MDI per le prime variabili ( $X_2 \approx 0.6$ , $X_3-X_4 \approx 0.3$ ), ma distribuisce importanza residua su un numero maggiore di variabili lontane ( $X_5-X_{12}$ tutte con barre visibili). Il vantaggio della Permutation rispetto alla MDI su dati con correlazione non emerge chiaramente in questa singola esecuzione — lo vedremo in modo quantitativo nell'Esperimento 5 (precision e recall su 40 run) |
| <b>Conditional</b> | Concentra quasi tutta l'importanza su $X_1$ ( $\approx 0.95$ ); $X_2$ scende a $\approx 0.35$ , tutto il resto cade vicino a zero. È l'unico metodo che isola la variabile causale in modo netto in questa singola esecuzione                                                                                                                                                                                                                |

## 6 Esperimento 4 — Stability Selection

### 6.1 Algoritmo e garanzie teoriche

La Stability Selection risponde alla domanda: **questa variabile viene selezionata sempre, o solo ogni tanto per caso?** Una variabile informativa dovrebbe emergere in qualsiasi sottocampione del dataset; una variabile rumore verrà selezionata solo sporadicamente.

Il procedimento con LASSO come selettore base:

1. Si generano  $B = 100$  sotto-campioni di dimensione  $n/2$  senza reimmissione.
2. Su ciascuno si stima un percorso LASSO e si registrano le variabili selezionate.
3. La probabilità di selezione  $\hat{\Pi}_j$  è la frazione di sotto-campioni con  $X_j$  selezionata.

Con  $\pi_{\text{thr}} = 0.75$  e  $\text{PFER} = 1$ , il metodo garantisce al massimo **1 falso positivo atteso** tra tutte le variabili selezionate. Il pacchetto `stabs` usa la versione Shah & Samworth (2013) con assunzione di unimodalità, che fornisce garanzie più stringenti rispetto alla formulazione originale.

```
X_stabs <- as.matrix(df_corr[, paste0("X", 1:p)])
Y_stabs <- df_corr$Y

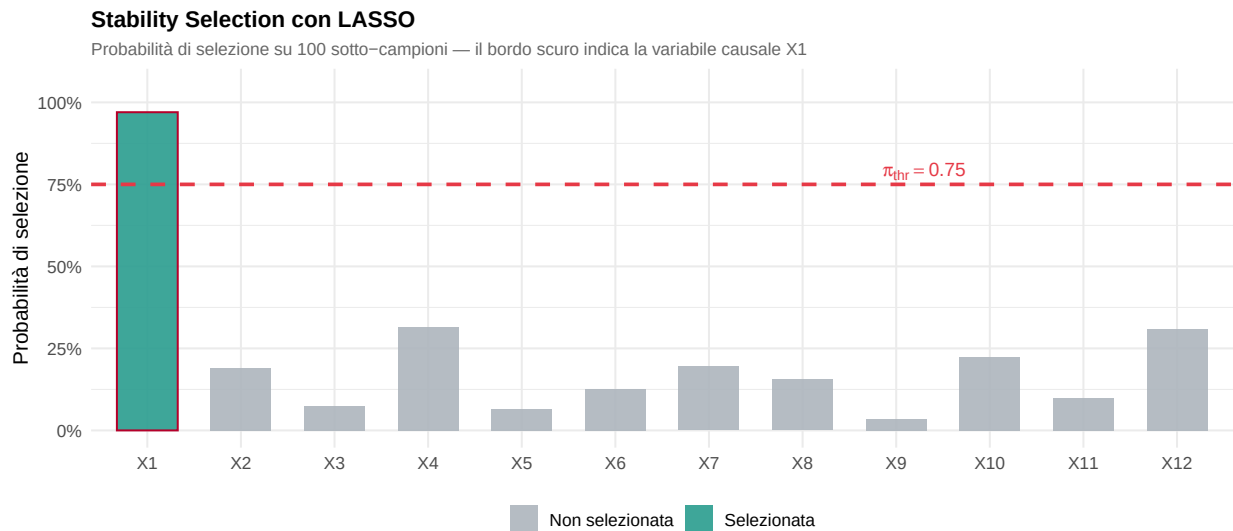
set.seed(404)
stab_lasso <- stabsel(
  x      = X_stabs,
  y      = Y_stabs,
  fitfun = glmnet.lasso,
  cutoff = 0.75,
  PFER   = 1,
  B      = 100,
  sampling.type = "SS"
)
print(stab_lasso)

## Stability Selection with unimodality assumption
##
## Selected variables:
## X1
## 1
##
## Selection probabilities:
##   X9   X5   X3  X11   X6   X8   X2   X7  X10  X12   X4   X1
## 0.035 0.065 0.075 0.100 0.125 0.155 0.190 0.195 0.225 0.310 0.315 0.970
##
## ---
## Cutoff: 0.75; q: 3; PFER (*): 0.758
##   (*) or expected number of low selection probability variables
## PFER (specified upper bound): 1
## PFER corresponds to signif. level 0.0632 (without multiplicity adjustment)

sel_probs <- stab_lasso$max[paste0("X", 1:p)]

stab_df <- data.frame(
  Variable = factor(paste0("X", 1:p), levels = paste0("X", 1:p)),
  Prob     = sel_probs,
  Selected = sel_probs >= 0.75,
  Causale  = paste0("X", 1:p) == "X1"
)
```

```
ggplot(stab_df, aes(x = Variable, y = Prob, fill = Selected, color = Causale)) +
  geom_col(width = 0.65, alpha = 0.9) +
  geom_hline(yintercept = 0.75, linetype = "dashed", color = "#E63946", linewidth = 1) +
  annotate("text", x = 9.3, y = 0.79,
    label = expression(pi[thr] == 0.75),
    color = "#E63946", size = 3.8) +
  scale_fill_manual(
    values = c("TRUE" = "#2A9D8F", "FALSE" = "#ADB5BD"),
    labels = c("TRUE" = "Selezionata", "FALSE" = "Non selezionata"),
    name = NULL
  ) +
  scale_color_manual(
    values = c("TRUE" = "#B5002A", "FALSE" = "transparent"),
    guide = "none"
  ) +
  scale_y_continuous(limits = c(0, 1.05), labels = percent_format(accuracy = 1)) +
  labs(
    title = "Stability Selection con LASSO",
    subtitle = "Probabilità di selezione su 100 sotto-campioni — il bordo scuro indica la variabile causale X1",
    x = NULL, y = "Probabilità di selezione"
  )
)
```



**Risultato:**  $X_1$  raggiunge  $\hat{\Pi}_1 = 0.97$ : viene selezionata nel 97% dei sotto-campioni, ben al di sopra della soglia  $\pi_{\text{thr}} = 0.75$ . Tutte le altre variabili restano sotto il 35% — con  $X_4$  e  $X_{12}$  che mostrano le probabilità più alte tra le variabili rumore (rispettivamente ~32% e ~31%) senza avvicinarsi alla soglia. Il loro segnale scompare non appena si cambia il campione, confermando che si trattava solo di un artefatto della correlazione con  $X_1$ .

**Nota sull'output di stabs:** il valore PFER (\*) : 0.758 riportato nell'output non è il numero di falsi positivi tra le variabili *selezionate*: la riga con (\*) indica il numero atteso di variabili a *bassa* probabilità di selezione erroneamente incluse, una metrica interna al pacchetto. Il bound formale sui falsi positivi tra le variabili selezionate è controllato dal parametro  $\text{PFER} = 1$  specificato nella chiamata, che garantisce  $E[|\hat{S}^{\text{stable}} \cap S^c|] \leq 1$ .

La Stability Selection produce una selezione **univoca e corretta**, con garanzia formale sul numero di falsi positivi.

---

## 7 Esperimento 5 — Confronto quantitativo: precision e recall

### 7.1 Setup con struttura a blocchi

Misuriamo quantitativamente le prestazioni di selezione su  $N = 40$  ripetizioni, calcolando per ogni metodo:

- **Precision** = quante delle variabili selezionate sono davvero causali?
- **Recall** = quante delle variabili causali vengono trovate?

Il dataset ha  $p = 30$  predittori in **3 blocchi da 10**, con correlazione intra-blocco  $\rho_{\text{intra}} = 0.80$  e correlazione inter-blocco  $\rho_{\text{inter}} = 0.30$ . La correlazione inter-blocco non nulla rende il problema realisticamente difficile: le variabili causali di blocchi diversi non sono più indipendenti tra loro. Le variabili causali sono  $X_1$ ,  $X_{11}$ ,  $X_{21}$  (la prima di ogni blocco), con segnale debole  $\beta = 1$  e  $n = 100$  osservazioni.

```
p6 <- 30
n6 <- 100      # n piccolo: il problema è difficile
rho6_intra <- 0.80
rho6_inter <- 0.30 # correlazione inter-blocco non nulla
beta6_val <- 1    # segnale debole
true_vars6 <- c("X1", "X11", "X21")

beta6 <- rep(0, p6)
beta6[c(1, 11, 21)] <- beta6_val

# Matrice di correlazione a blocchi con correlazione inter-blocco
Sigma6 <- matrix(rho6_inter, p6, p6)      # base: rho_inter ovunque
for (b in 0:2) {
  idx <- (b * 10 + 1):((b + 1) * 10)
  Sigma6[idx, idx] <- rho6_intra          # intra-blocco più alta
}
diag(Sigma6) <- 1

set.seed(600)
X6 <- mvrnorm(n6, mu = rep(0, p6), Sigma = Sigma6)
colnames(X6) <- paste0("X", 1:p6)
Y6 <- as.numeric(X6 %*% beta6 + rnorm(n6))
df6 <- data.frame(Y = Y6, X6)
```

### 7.2 Precision e Recall su 40 run

```
n_rep6 <- 40
k6 <- 3    # selezioniamo le top-3 variabili

pr_results6 <- lapply(seq_len(n_rep6), function(i) {
  set.seed(i * 17)
  rf6 <- randomForest(Y ~ ., data = df6, ntree = 500, importance = TRUE)

  top_mdi <- names(sort(importance(rf6, type = 2)[, 1], decreasing = TRUE))[1:k6]
  top_perm <- names(sort(importance(rf6, type = 1)[, 1], decreasing = TRUE))[1:k6]

  data.frame(
    run      = i,
    MDI_prec = mean(top_mdi %in% true_vars6),
```

```

    Perm_prec = mean(top_perm %in% true_vars6),
    MDI_rec   = sum(true_vars6 %in% top_mdi) / length(true_vars6),
    Perm_rec   = sum(true_vars6 %in% top_perm) / length(true_vars6)
  )
})
pr_df6 <- do.call(rbind, pr_results6)

# Verifica numerica: medie e SD su 40 run
cat("Medie su 40 run:\n")

## Medie su 40 run:
print(round(colMeans(pr_df6[, -1]), 3))

##   MDI_prec Perm_prec   MDI_rec Perm_rec
##   0.667     0.750     0.667     0.750

cat("\nDeviazioni standard:\n")

##
## Deviazioni standard:
print(round(apply(pr_df6[, -1], 2, sd), 3))

##   MDI_prec Perm_prec   MDI_rec Perm_rec
##   0.000     0.146     0.000     0.146

pr_long6 <- pr_df6 %>%
  pivot_longer(-run, names_to = "key", values_to = "value") %>%
  mutate(
    Metodo = ifelse(grepl("MDI", key), "MDI", "Permutation"),
    Metrica = ifelse(grepl("prec", key), "Precision", "Recall")
  )

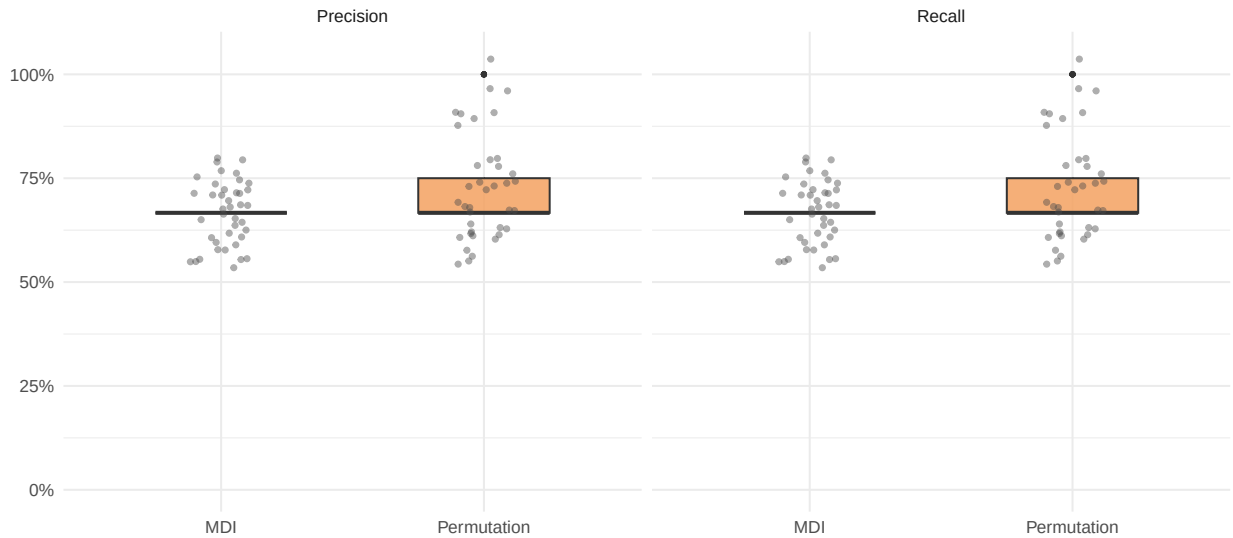
ggplot(pr_long6, aes(x = Metodo, y = value, fill = Metodo)) +
  geom_boxplot(alpha = 0.85, width = 0.5, outlier.size = 1) +
  geom_jitter(width = 0.12, size = 1.2, alpha = 0.45, color = "grey30") +
  facet_wrap(~ Metrica) +
  scale_fill_manual(values = c(MDI = "#E63946", Permutation = "#F4A261"),
                    guide = "none") +
  scale_y_continuous(limits = c(0, 1.05),
                    labels = percent_format(accuracy = 1)) +
  labs(
    title = "Precision e Recall su 40 run indipendenti",
    subtitle = paste0("3 variabili causali su ", p6,
                      " - blocchi \u03c11=", rho6_intra,
                      ", inter-blocco \u03c11=", rho6_inter,
                      ", \u03b22=", beta6_val, ", n=", n6),
    x = NULL, y = NULL
  )

```



### Precision e Recall su 40 run indipendenti

3 variabili causali su 30 — blocchi  $\rho=0.8$ , inter-blocco  $\rho=0.3$ ,  $\beta=1$ ,  $n=100$



**Interpretazione:** Con segnale debole,  $n$  piccolo e correlazione sia intra- che inter-blocco, la differenza tra MDI e Permutation Importance emerge chiaramente sia in precision che in recall.

La MDI mostra  $SD=0$  su tutti i 40 run: non è instabile, è **sistematicamente bloccata** su una selezione errata. I punti si concentrano su soli due valori discreti — 33% e 67% di 3 variabili causali, cioè 1 o 2 causali trovate su 3 — e il risultato non cambia mai tra un run e l'altro. Il bias da cardinalità e dalla correlazione intra-blocco porta MDI a selezionare ripetutamente le stesse variabili rumore del blocco (es.  $X_2$  al posto di  $X_1$ ), producendo errori sistematici e non casuali.

La Permutation Importance ha mediana più alta (~75% vs ~67% di MDI) e variabilità reale tra run ( $SD=0.146$ ): in molte esecuzioni riesce a trovare tutte e 3 le causali (100%), in altre ne manca una. Questo trade-off — più accurata in media, ma con variabilità residua — è il comportamento atteso di un metodo meno distorto ma non esente da errori con  $n$  piccolo. Il rimando promesso nell'Esperimento 3 è confermato: su questa singola esecuzione il vantaggio della Permutation non era visibile, ma su 40 run il divario sistematico in precision e recall è netto.

## 8 Esperimento 6 — SHAP values: l'alternativa moderna

### 8.1 Cos'è SHAP

**SHAP** (SHapley Additive exPlanations, Lundberg & Lee 2017) è lo standard de facto per l'interpretabilità dei modelli di machine learning. L'idea viene dalla teoria dei giochi cooperativi: ogni variabile riceve come "importanza" il suo contributo marginale medio alle previsioni, calcolato su tutte le possibili coalizioni di variabili presenti o assenti.

A differenza di MDI e Permutation Importance, SHAP produce un **valore per ogni singola osservazione**: si vede non solo *quanto* conta una variabile in media, ma *come* influenza ogni singola previsione e *in che direzione*.

**Nota tecnica:** qui usiamo **fastshap**, che stima i valori SHAP tramite campionamento Monte Carlo ( $nsim = 100$ ). È un'approssimazione: con molte variabili correlate o con  $nsim$  basso i valori possono essere rumorosi. Per stime esatte su alberi si possono usare i pacchetti **treeshap** o **shapviz + ranger**.

```

set.seed(707)
pfun <- function(object, newdata) predict(object, newdata = newdata)

shap_vals <- fastshap::explain(
  rf_main,
  X          = df_corr[, paste0("X", 1:p)],
  pred_wrapper = pfun,
  nsim       = 100
)

shap_mean_imp <- colMeans(abs(shap_vals))
var_order_shap <- names(sort(shap_mean_imp, decreasing = TRUE))

shap_long <- as.data.frame(shap_vals) %>%
  mutate(obs = row_number()) %>%
  pivot_longer(-obs, names_to = "Variable", values_to = "SHAP")

feat_long <- df_corr[, paste0("X", 1:p)] %>%
  mutate(obs = row_number()) %>%
  pivot_longer(-obs, names_to = "Variable", values_to = "FeatVal") %>%
  group_by(Variable) %>%
  mutate(FeatVal_sc = (FeatVal - min(FeatVal)) /
          (max(FeatVal) - min(FeatVal))) %>%
  ungroup()

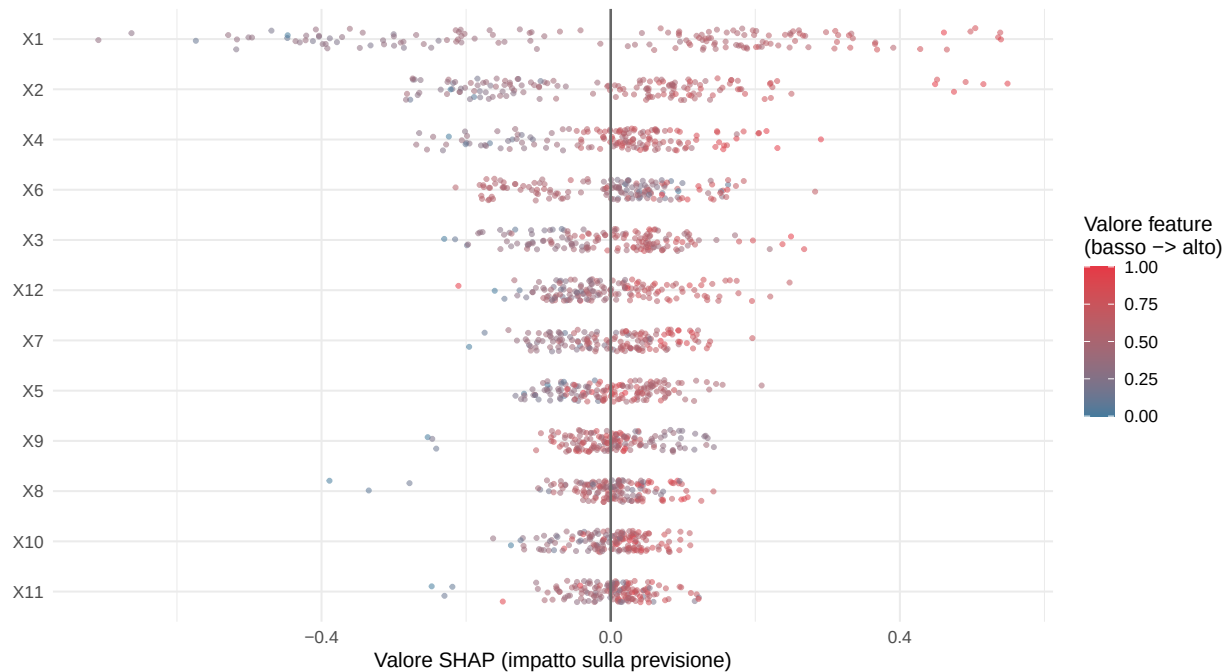
shap_bee <- left_join(shap_long, feat_long, by = c("obs", "Variable")) %>%
  mutate(Variable = factor(Variable, levels = rev(var_order_shap)))

ggplot(shap_bee, aes(x = SHAP, y = Variable, color = FeatVal_sc)) +
  geom_jitter(height = 0.22, size = 0.9, alpha = 0.55) +
  geom_vline(xintercept = 0, color = "grey40", linewidth = 0.7) +
  scale_color_gradient(low = "#457B9D", high = "#E63946",
    name = "Valore feature\n(basso \u2192 alto)") +
  labs(
    title = "SHAP Beeswarm Plot \u2014 Random Forest su dati AR(1)",
    subtitle = "Ogni punto = un'osservazione | colore = valore della feature | posizione x = impatto su",
    x = "Valore SHAP (impatto sulla previsione)", y = NULL
  ) +
  theme(legend.position = "right")

```

### SHAP Beeswarm Plot — Random Forest su dati AR(1)

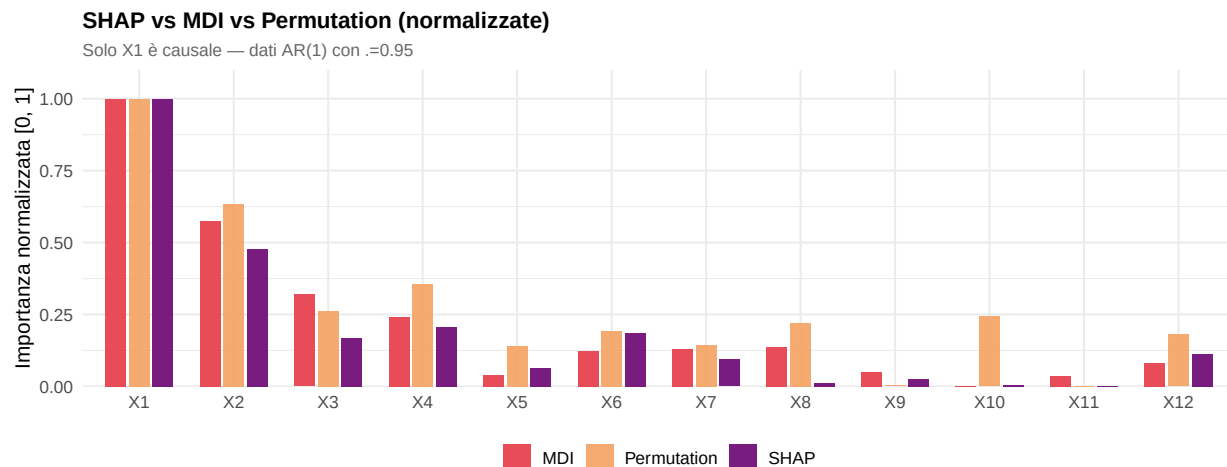
Ogni punto = un'osservazione | colore = valore della feature | posizione x = impatto sulla previsione



```
imp_comp <- data.frame(
  Variable = paste0("X", 1:p),
  SHAP     = normalize(shap_mean_imp),
  MDI      = normalize(imp_mdi_main),
  Permutation = normalize(imp_mda_main)
) %>%
pivot_longer(-Variable, names_to = "Metodo", values_to = "Importanza") %>%
mutate(
  Variable = factor(Variable, levels = paste0("X", 1:p)),
  Metodo   = factor(Metodo, levels = c("MDI", "Permutation", "SHAP"))
)

pal_comp <- c(MDI = "#E63946", Permutation = "#F4A261", SHAP = "#6A0572")

ggplot(imp_comp, aes(x = Variable, y = Importanza, fill = Metodo)) +
  geom_col(position = position_dodge(0.75), width = 0.65, alpha = 0.9) +
  scale_fill_manual(values = pal_comp) +
  scale_y_continuous(limits = c(0, 1.1), expand = c(0, 0)) +
  labs(
    title = "SHAP vs MDI vs Permutation (normalizzate)",
    subtitle = paste0("Solo X1 \u00e8 causale \u2014 dati AR(1) con \u03c1=", rho),
    x = NULL, y = "Importanza normalizzata [0, 1]", fill = NULL
  )
```



**Interpretazione:** I valori SHAP si comportano in modo simile alla Permutation Importance:  $X_1$  domina chiaramente, mentre le variabili correlate ricevono importanza residua decrescente. Il beeswarm aggiunge però un'informazione cruciale: valori alti di  $X_1$  (rosso) producono SHAP positivi — aumentano la previsione — e valori bassi (blu) la abbassano. Questo è coerente con il coefficiente positivo  $\beta_1 = 0.8$  del modello generativo. Per le variabili rumore, i punti sono sparsi simmetricamente attorno allo zero: nessuna direzione d'effetto stabile.

## 9 Esperimento 7 — L'effetto del rapporto $n/p$

### 9.1 Come cambia la qualità della selezione al variare della dimensione del campione?

In molte applicazioni reali — soprattutto in ambito genomico — il numero di osservazioni  $n$  è molto inferiore al numero di variabili  $p$ . Questo aggrava tutti i problemi visti finora: con pochi dati, il bias della MDI è amplificato e la varianza delle stime di importanza cresce.

Fissiamo  $p = 15$  predittori con struttura AR(1) a  $\rho = 0.70$  e 3 variabili causali ( $X_1, X_6, X_{11}$ ), con  $\beta = 2$ , e facciamo variare  $n$  da 50 a 1500. Per ogni valore di  $n$  ripetiamo 50 volte e misuriamo la precision media del top-3. La banda rappresenta  $\pm 1$  SD. Usiamo 50 ripetizioni (invece di 30) per stabilizzare le stime nei regimi a  $n$  piccolo, dove la variabilità è più alta.

```
p8 <- 15
rho8 <- 0.70
true8 <- c("X1", "X6", "X11")
beta8 <- rep(0, p8)
beta8[c(1, 6, 11)] <- 2
Sigma8 <- toeplitz(rho8^(0:(p8 - 1)))
n_vals8 <- c(50, 100, 200, 400, 800, 1500)
n_rep8 <- 50 # aumentato da 30 per stabilizzare le stime a n piccolo
```

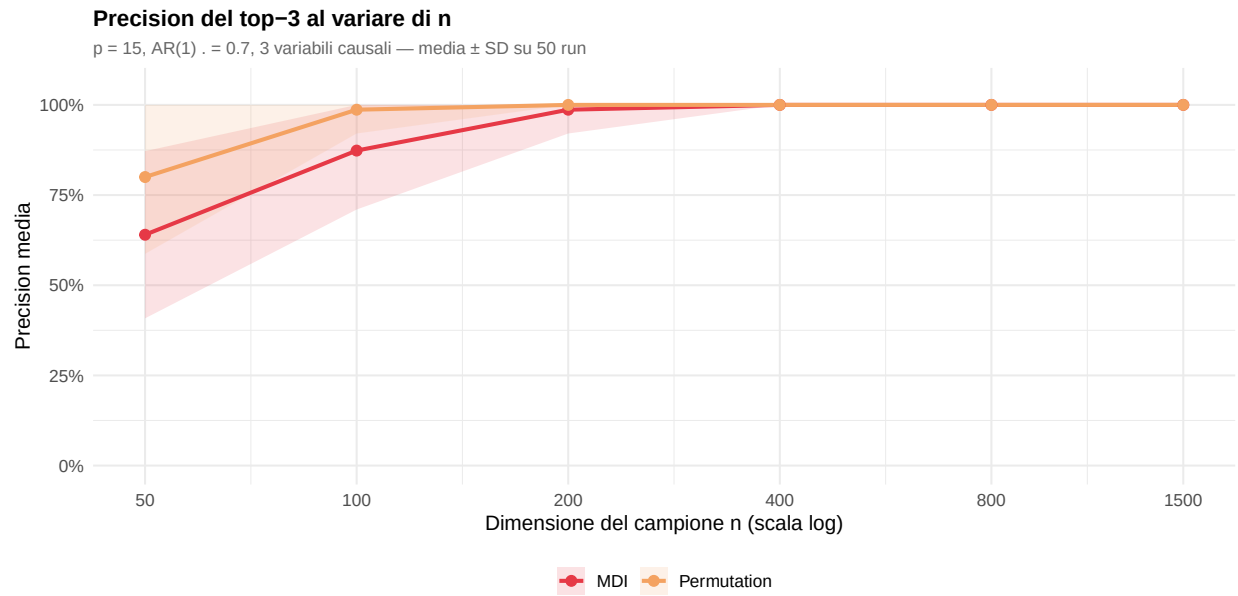
```
results8 <- lapply(n_vals8, function(n8) {
  pr <- sapply(seq_len(n_rep8), function(i) {
    set.seed(i * 31)
    X8 <- mvrnorm(n8, rep(0, p8), Sigma8)
    colnames(X8) <- paste0("X", 1:p8)
    Y8 <- as.numeric(X8 %*% beta8 + rnorm(n8))
    df8 <- data.frame(Y = Y8, X8)
```

```

rf8 <- randomForest(Y ~ ., data = df8, ntree = 400, importance = TRUE)
top_mdi <- names(sort(importance(rf8, type = 2)[, 1], dec = TRUE))[1:3]
top_perm <- names(sort(importance(rf8, type = 1)[, 1], dec = TRUE))[1:3]
c(MDI = mean(top_mdi %in% true8),
  Permutation = mean(top_perm %in% true8))
})
data.frame(
  n = n8,
  Metodo = c("MDI", "Permutation"),
  Precision = rowMeans(pr),
  Prec_sd = apply(pr, 1, sd)
)
})
res8_df <- do.call(rbind, results8)

ggplot(res8_df, aes(x = n, y = Precision, color = Metodo, group = Metodo)) +
  geom_ribbon(aes(ymin = pmax(0, Precision - Prec_sd),
    ymax = pmin(1, Precision + Prec_sd),
    fill = Metodo), alpha = 0.15, color = NA) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.5) +
  scale_color_manual(values = c(MDI = "#E63946", Permutation = "#F4A261")) +
  scale_fill_manual(values = c(MDI = "#E63946", Permutation = "#F4A261")) +
  scale_x_log10(breaks = n_vals8, labels = n_vals8) +
  scale_y_continuous(limits = c(0, 1.05),
    labels = percent_format(accuracy = 1)) +
  labs(
    title = "Precision del top-3 al variare di n",
    subtitle = paste0("p = ", p8, ", AR(1) \u03c1 = ", rho8,
      ", 3 variabili causali \u2014 media \u00b1 SD su ",
      n_rep8, " run"),
    x = "Dimensione del campione n (scala log)",
    y = "Precision media", color = NULL, fill = NULL
  )

```



**Interpretazione:** La Permutation Importance è sistematicamente superiore alla MDI a parità di  $n$ , con un divario massimo nelle regioni a campione piccolo dove il bias MDI si somma alla varianza da campionamento. Entrambi i metodi convergono verso la precisione massima per  $n$  grandi, ma la Permutation vi arriva prima. Questo è particolarmente critico in genomica, dove  $n$  è spesso fissato dai costi sperimentali e non può essere aumentato arbitrariamente.

## 10 Esperimento 8 — Dati ad alta dimensionalità ( $p \gg n$ )

### 10.1 Il caso genomico: molte più variabili che osservazioni

In genomica è comune avere  $p \gg n$ : migliaia di geni misurati su poche decine di pazienti. In questo regime i problemi visti finora si esasperano: la MDI diventa instabile perché centinaia di variabili rumore competono con i segnali reali su pochissimi dati.

Simuliamo  $p = 300$  predittori **incorrelati** con  $n = 80$  osservazioni e 5 variabili causali sparse, con segnale moderato ( $\beta = 1.5$ ). Il caso incorrelato è il più favorevole possibile per la MDI; con segnale/rumore limitato e  $p \gg n$  l'instabilità è comunque rilevante.

```
p9 <- 300
n9 <- 80
true9 <- c(1, 60, 120, 180, 240)
true_names9 <- paste0("X", true9)

set.seed(900)
X9 <- matrix(rnorm(n9 * p9), nrow = n9, ncol = p9)
colnames(X9) <- paste0("X", 1:p9)
beta9 <- rep(0, p9)
beta9[true9] <- 1.5
Y9 <- as.numeric(X9 %*% beta9 + rnorm(n9))
df9 <- data.frame(Y = Y9, X9)

set.seed(901); rf9a <- randomForest(Y ~ ., data = df9, ntree = 1000, importance = TRUE)
set.seed(999); rf9b <- randomForest(Y ~ ., data = df9, ntree = 1000, importance = TRUE)
```

```

rank9a <- sort(rank(-importance(rf9a, type = 2)[, 1]))
rank9b <- sort(rank(-importance(rf9b, type = 2)[, 1]))
top_k9 <- 20
top9a <- names(rank9a)[1:top_k9]
top9b <- names(rank9b)[1:top_k9]

# Statistiche di concordanza tra i due run
overlap_n <- length(intersect(top9a, top9b))
jaccard <- overlap_n / length(union(top9a, top9b))
recall9a <- sum(top9a %in% true_names9)
recall9b <- sum(top9b %in% true_names9)

cat(sprintf("Overlap top-%d tra i due run: %d/%d (%.0f%%)\n",
            top_k9, overlap_n, top_k9,
            100 * overlap_n / top_k9))

## Overlap top-20 tra i due run: 17/20 (85%)

cat(sprintf("Jaccard top-%d: %.2f\n", top_k9, jaccard))

## Jaccard top-20: 0.74

cat(sprintf("Variabili causali in top-%d: Run1 = %d/5 | Run2 = %d/5\n",
            top_k9, recall9a, recall9b))

## Variabili causali in top-20: Run1 = 5/5 | Run2 = 5/5

cat(sprintf(
  "\nNota: le variabili causali sono recuperate, ma sono accompagnate da %d (Run1) e %d (Run2)\n",
  top_k9 - recall9a, top_k9 - recall9b))

##
## Nota: le variabili causali sono recuperate, ma sono accompagnate da 15 (Run1) e 15 (Run2)

cat("variabili rumore diverse nei due run. Un analista ignaro non sa quali sono le causali.\n")

## variabili rumore diverse nei due run. Un analista ignaro non sa quali sono le causali.

top40_9 <- names(rank9a)[1:40]
imp9a_show <- importance(rf9a, type = 2)[top40_9, 1]
imp9b_show <- importance(rf9b, type = 2)[top40_9, 1]

plot9_df <- data.frame(
  Variable = factor(top40_9, levels = top40_9),
  `Run 1` = imp9a_show,
  `Run 2` = imp9b_show[top40_9],
  check.names = FALSE
) %>%
  pivot_longer(-Variable, names_to = "Run", values_to = "MDI") %>%
  mutate(Causale = Variable %in% true_names9)

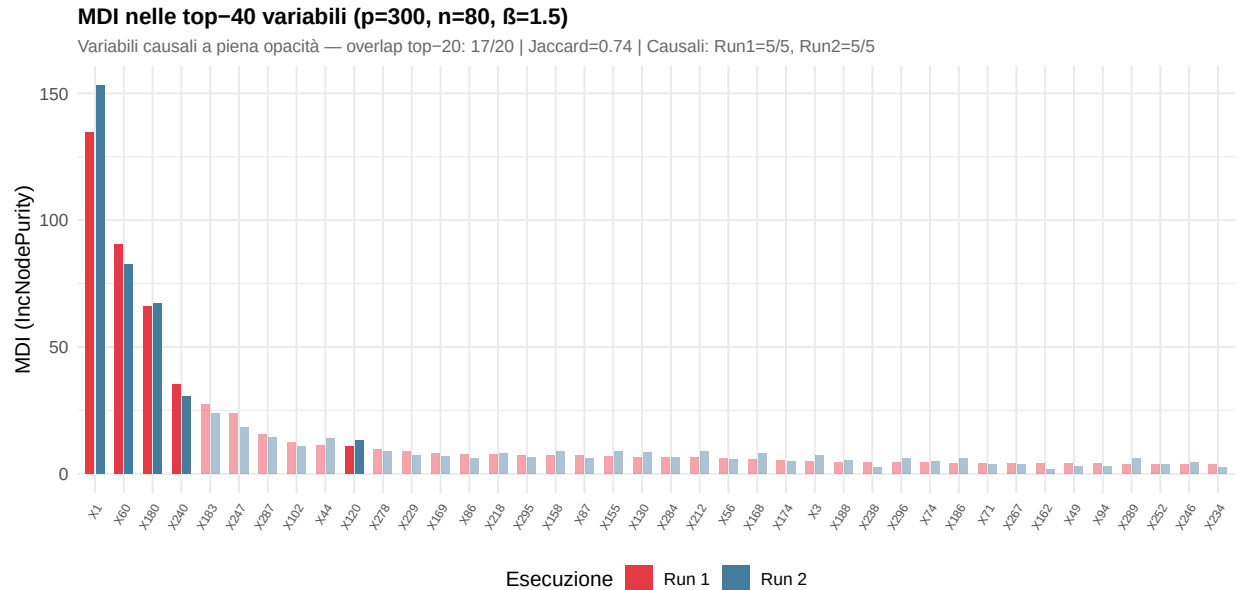
ggplot(plot9_df, aes(x = Variable, y = MDI, fill = Run, alpha = Causale)) +
  geom_col(position = position_dodge(0.7), width = 0.6) +
  scale_fill_manual(values = c("Run 1" = "#E63946", "Run 2" = "#457B9D")) +
  scale_alpha_manual(values = c("TRUE" = 1, "FALSE" = 0.45), guide = "none") +
  labs(
    title = paste0("MDI nelle top-40 variabili (p=", p9, ", n=", n9, ", \u03b2=1.5)"),

```

```

subtitle = paste0("Variabili causali a piena opacit\u00e0 \u2014 overlap top-",
  top_k9, ":", overlap_n, "/", top_k9,
  " | Jaccard=", round(jaccard, 2),
  " | Causali: Run1=", recall9a, "/5, Run2=", recall9b, "/5"),
x = NULL, y = "MDI (IncNodePurity)", fill = "Esecuzione"
) +
theme(axis.text.x = element_text(angle = 60, hjust = 1, size = 7))

```



**Interpretazione:** Con  $p \gg n$  e segnale moderato si osservano due effetti distinti. Da un lato, le 5 variabili causali (barre a piena opacità) vengono recuperate in entrambi i run grazie al segnale  $\beta = 1.5$  — il che è il caso più favorevole, con predittori incorrelati. Dall'altro, i 15 posti rimanenti nella top-20 sono occupati da variabili rumore diverse nei due run (Jaccard  $\approx 0.74$ ): un analista che non conosce la verità e lavora con la top-20 si troverebbe con un insieme contaminato da 15 falsi positivi, e l'insieme stesso cambierebbe ad ogni esecuzione. In presenza di correlazione tra predittori — come in dati genomici reali — la situazione peggiorerebbe ulteriormente, rendendo Stability Selection e Conditional Importance essenziali.

## 11 Approfondimento — Il metodo Boruta

### 11.1 L'idea: confrontare ogni variabile con la propria “ombra”

**Boruta** (Kursa & Rudnicki, 2010) affronta il bias da cardinalità confrontando ogni variabile non con un valore assoluto, ma con una sua **copia casuale (ombra)**: i valori della variabile originale vengono rimescolati, distruggendo il segnale. Per costruzione un'ombra non può essere informativa. Il Random Forest viene addestrato su originali + ombre, e per ogni iterazione si registra il massimo delle importanze ombra (*shadowMax*). Una variabile è dichiarata:

- **Confermata:** supera sistematicamente shadowMax (test binomiale).
- **Rifiutata:** rimane sistematicamente al di sotto.
- **Tentativa:** il risultato è incerto.

Il vantaggio chiave è che ogni variabile è confrontata con un'ombra **della stessa cardinalità**: il bias da cardinalità si cancella nella comparazione.



```

set.seed(808)
boruta_res <- Boruta::Boruta(
  Y ~ ., data = df_corr,
  doTrace = 0,
  maxRuns = 150,
  ntree = 500
)
print(boruta_res)

## Boruta performed 117 iterations in 2.00544 secs.
## 12 attributes confirmed important: X1, X10, X11, X12, X2 and 7 more;
## No attributes deemed unimportant.

boruta_stats <- attStats(boruta_res)
boruta_stats$Variable <- rownames(boruta_stats)

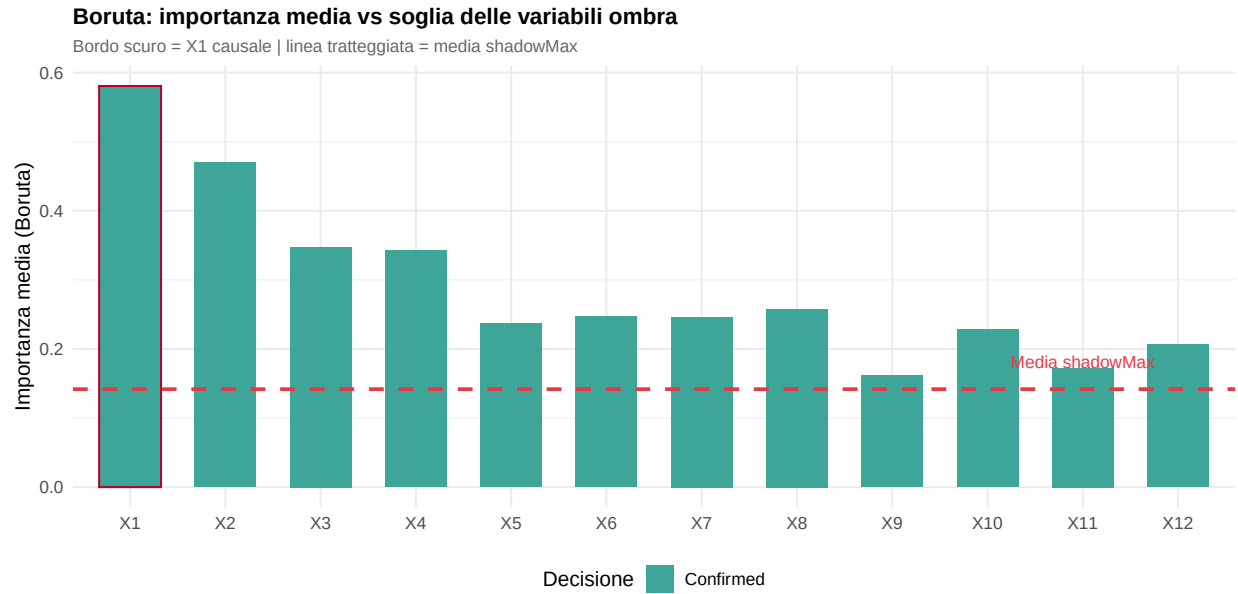
shadow_thr <- mean(boruta_res$ImpHistory[, "shadowMax"], na.rm = TRUE)

boruta_plot_df <- boruta_stats %>%
  mutate(
    Variable = factor(Variable, levels = paste0("X", 1:p)),
    Decisione = factor(decision,
      levels = c("Confirmed", "Tentative", "Rejected")),
    Causale = Variable == "X1"
  )

pal_boruta <- c(Confirmed = "#2A9D8F",
  Tentative = "#F4A261",
  Rejected = "#ADB5BD")

ggplot(boruta_plot_df,
  aes(x = Variable, y = meanImp, fill = Decisione, color = Causale)) +
  geom_col(width = 0.65, alpha = 0.9) +
  geom_hline(yintercept = shadow_thr, linetype = "dashed",
    color = "#E63946", linewidth = 1) +
  annotate("text", x = p - 1, y = shadow_thr + 0.04,
    label = "Media shadowMax", color = "#E63946", size = 3.5) +
  scale_fill_manual(values = pal_boruta) +
  scale_color_manual(values = c("TRUE" = "#B5002A", "FALSE" = "transparent"),
    guide = "none") +
  labs(
    title = "Boruta: importanza media vs soglia delle variabili ombra",
    subtitle = "Bordo scuro = X1 causale | linea tratteggiata = media shadowMax",
    x = NULL, y = "Importanza media (Boruta)", fill = "Decisione"
  )

```



**Interpretazione:**  $X_1$  supera sistematicamente la soglia shadowMax e viene confermata. Le variabili correlate ( $X_2$ ,  $X_3$ , ...) vengono anch'esse confermate con  $\rho = 0.95$ : questo non è un difetto di Boruta, ma riflette il fatto che variabili fortemente correlate con la causa portano informazione predittiva reale, anche se non causale. Boruta non distingue causa da proxy correlato — è necessario complementarlo con la Conditional Importance o la Stability Selection quando l'obiettivo è l'identificazione causale. La tabella riepilogativa riflette questa distinzione.

## 12 Riepilogo e confronto sintetico

```
X1_MDI_top <- which.max(imp_mdi_main) == 1
X1_MDA_top <- which.max(imp_mda_main) == 1
X1_COND_top <- which.max(imp_cond_main[paste0("X", 1:p)]) == 1
X1_STAB_ok <- stab_df$Prob[1] >= 0.75 && all(stab_df$Prob[-1] < 0.75)
X1_SHAP_top <- which.max(shap_mean_imp) == 1

# Boruta: X1 confermata? quante altre variabili confermate?
X1_boruta_confirmed <- boruta_res$finalDecision["X1"] == "Confirmed"
n_boruta_confirmed <- sum(boruta_res$finalDecision == "Confirmed")
boruta_label <- if (X1_boruta_confirmed) {
  if (n_boruta_confirmed == 1) "S\u00ec (solo X1)" else
  paste0("S\u00ec (+ ", n_boruta_confirmed - 1, " proxy)")
} else "No"

# MDI: identificata, ma instabile tra run (nota esplicita)
mdi_label <- if (X1_MDI_top) "S\u00ec (instabile)" else "No"

tick_cross <- function(x) ifelse(x, "S\u00ec", "No")

summary_df <- data.frame(
  Metodo = c(
    "MDI (randomForest)",
```

```

    "Permutation Imp. (randomForest)",
    "Conditional Imp. (cforest)",
    "Stability Selection (LASSO)",
    "SHAP (fastshap)",
    "Boruta"
  ),
  Bias_card = c("S\u00ec", "No", "No", "No", "No", "No"),
  Instabile = c("S\u00ec", "Parzialmente", "No", "No", "Parzialmente", "No"),
  X1_corretta = c(
    mdi_label,
    tick_cross(X1_MDA_top),
    tick_cross(X1_COND_top),
    tick_cross(X1_STAB_ok),
    tick_cross(X1_SHAP_top),
    boruta_label
  ),
  Garanzia_FP = c("No", "No", "No", "S\u00ec (PFER)", "No", "No"),
  Interp_obs = c("No", "No", "No", "No", "S\u00ec", "No"),
  Costo = c("Basso", "Medio", "Alto*", "Medio", "Alto**", "Medio")
)

knitr::kable(
  summary_df,
  col.names = c("Metodo", "Bias cardinalit\u00e0",
    "Instabile con correlazione",
    "X1 identificata",
    "Garanzia falsi positivi",
    "Interpret. per osservazione",
    "Costo computazionale"),
  align = "lcccccc",
  caption = paste0(
    "Confronto tra metodi di variable importance. ",
    "*cforest usa conditional inference trees, algoritmo di splitting diverso da randomForest. ",
    "**fastshap \u00e8 un'approssimazione Monte Carlo; per stime esatte usare treeshap o shapviz+ranger
  )
)

```

Table 1: Confronto tra metodi di variable importance. \*cforest usa conditional inference trees, algoritmo di splitting diverso da randomForest. \*\*fastshap è un'approssimazione Monte Carlo; per stime esatte usare treeshap o shapviz+ranger.

| Metodo                          | Bias cardinalità | Instabile con correlazione | X1 identificata | Garanzia falsi positivi | Interpret. per osservazione | Costo computazionale |
|---------------------------------|------------------|----------------------------|-----------------|-------------------------|-----------------------------|----------------------|
| MDI (randomForest)              | Sì               | Sì                         | Sì (instabile)  | No                      | No                          | Basso                |
| Permutation Imp. (randomForest) | No               | Parzialmente               | Sì              | No                      | No                          | Medio                |
| Conditional Imp. (cforest)      | No               | No                         | Sì              | No                      | No                          | Alto*                |

| Metodo                      | Bias cardinalità | Instabile con correlazione | X1 identificata | Garanzia falsi positivi | Interpret. per osservazione | Costo computazionale |
|-----------------------------|------------------|----------------------------|-----------------|-------------------------|-----------------------------|----------------------|
| Stability Selection (LASSO) | No               | No                         | Sì              | Sì (PFER)               | No                          | Medio                |
| SHAP (fastshap)             | No               | Parzialmente               | Sì              | No                      | Sì                          | Alto**               |
| Boruta                      | No               | No                         | Sì (+ 11 proxy) | No                      | No                          | Medio                |

**Lettura della tabella:** Non esiste un metodo dominante su tutte le dimensioni. La scelta dipende dal contesto:

- Se il **costo computazionale** è un vincolo stringente e i predittori non sono ad alta cardinalità, la **Permutation Importance** è un buon compromesso.
- Se si lavora con predittori fortemente correlati e si vuole individuare la causa diretta, la **Conditional Importance** è la scelta più rigorosa — ricordando che si tratta di un modello diverso (cforest), non solo di un indice diverso su randomForest.
- Se serve una **garanzia formale sui falsi positivi** — tipica in ambito clinico o regolatorio — la **Stability Selection** è l'unica opzione.
- Se l'obiettivo è capire **come e quanto** ogni variabile influenza le singole previsioni (non solo la selezione globale), **SHAP** è lo strumento più ricco, con la cautela che **fastshap** fornisce stime approssimate.
- Se si vuole un metodo automatico e robusto al bias da cardinalità senza dover scegliere soglie, **Boruta** è pratico, ma in presenza di alta correlazione conferma anche i proxy causali insieme alla causa reale.

## 13 Riferimenti

- Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- Strobl, C., Boulesteix, A.-L., Zeileis, A., Hothorn, T. (2007). *Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution*. BMC Bioinformatics, 8, 25.
- Strobl, C., Boulesteix, A.-L., Kneib, T., Augustin, T., Zeileis, A. (2008). *Conditional Variable Importance for Random Forests*. BMC Bioinformatics, 9, 307.
- Meinshausen, N., Bühlmann, P. (2010). *Stability Selection*. Journal of the Royal Statistical Society: Series B, 72(4), 417–473.
- Shah, R. D., Samworth, R. J. (2013). *Variable selection with error control: Another look at Stability Selection*. Journal of the Royal Statistical Society: Series B, 75(1), 55–80.
- Hothorn, T., Hornik, K., Zeileis, A. (2006). *Unbiased Recursive Partitioning: A Conditional Inference Framework*. Journal of Computational and Graphical Statistics, 15(3).
- Lundberg, S. M., Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS), 30.
- Kursa, M. B., Rudnicki, W. R. (2010). *Feature Selection with the Boruta Package*. Journal of Statistical Software, 36(11), 1–13.

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
## LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Italian_Italy.utf8 LC_CTYPE=Italian_Italy.utf8
```

```

## [3] LC_MONETARY=Italian_Italy.utf8 LC_NUMERIC=C
## [5] LC_TIME=Italian_Italy.utf8
##
## time zone: Europe/Rome
## tzcode source: internal
##
## attached base packages:
## [1] parallel stats4 grid stats graphics grDevices utils
## [8] datasets methods base
##
## other attached packages:
## [1] fastshap_0.1.1 Boruta_10.0.0 scales_1.4.0
## [4] forcats_1.0.1 RColorBrewer_1.1-3 reshape2_1.4.5
## [7] patchwork_1.3.2 tidyr_1.3.2 dplyr_1.2.0
## [10] ggplot2_4.0.2 MASS_7.3-65 glmnet_5.0
## [13] Matrix_1.7-4 stabs_0.7-1 party_1.3-20
## [16] strucchange_1.5-4 sandwich_3.1-1 zoo_1.8-15
## [19] modeltools_0.2-24 mvtnorm_1.3-7 randomForest_4.7-1.2
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4 coin_1.4-3 shape_1.4.6.1 stringi_1.8.7
## [5] lattice_0.22-7 digest_0.6.39 magrittr_2.0.4 evaluate_1.0.5
## [9] iterators_1.0.14 fastmap_1.2.0 plyr_1.8.9 foreach_1.5.2
## [13] survival_3.8-3 multcomp_1.4-30 fru_0.0.7 purrr_1.2.1
## [17] TH.data_1.1-5 codetools_0.2-20 cli_3.6.5 rlang_1.1.7
## [21] splines_4.5.2 withr_3.0.2 yaml_2.3.12 otel_0.2.0
## [25] tools_4.5.2 vctrs_0.7.1 R6_2.6.1 matrixStats_1.5.0
## [29] lifecycle_1.0.5 stringr_1.6.0 libcoin_1.0-12 pkgconfig_2.0.3
## [33] pillar_1.11.1 gtable_0.3.6 glue_1.8.0 Rcpp_1.1.1
## [37] xfun_0.56 tibble_3.3.1 tidyselect_1.2.1 knitr_1.51
## [41] farver_2.1.2 htmltools_0.5.9 labeling_0.4.3 rmarkdown_2.30
## [45] compiler_4.5.2 S7_0.2.1

```